Agent Lifecycle

Nov 6, 2026 · @Ashutosh

An AI agent has two lifecycles: an engineering lifecycle that takes it from idea to production and back for improvement, and a run-time loop that every request passes through. Most production failures trace to a skipped stage in the first or a missing safeguard in the second.

1. Engineering lifecycle

The engineering lifecycle runs in eight stages, and stages 3 to 7 repeat for as long as the agent is in service.

1. **Define**: state the task, the users, what "done" looks like and what the agent must never do. Set measurable targets such as task completion rate, accuracy, cost per task and latency.
2. **Design**: choose the architecture. Decide whether a fixed workflow is enough or the model must choose its own steps, whether one agent or several are needed, which tools it gets, and where a human must approve.
3. **Build**: write the prompts, tools (or connect them through MCP servers), state schema and memory. In the LangChain stack that means `create_agent` for a standard loop or a LangGraph graph for custom control flow.
4. **Evaluate**: test against a dataset of realistic tasks before release. Check the final answer and the path taken: right tools, right order, no wasted steps.
5. **Deploy**: run on infrastructure that keeps state durable, scales, and supports long-running or background runs. Put guardrails in place before real users arrive.
6. **Monitor**: trace every run (model calls, tool calls, tokens, latency, errors) and collect user feedback.
7. **Improve**: turn production failures into new test cases, change prompts, tools or graph structure, then re-evaluate. This loops back to stage 3.
8. **Retire or version**: replace the agent with a new version, migrate or delete its stored memory and threads, and revoke its credentials and tool access.

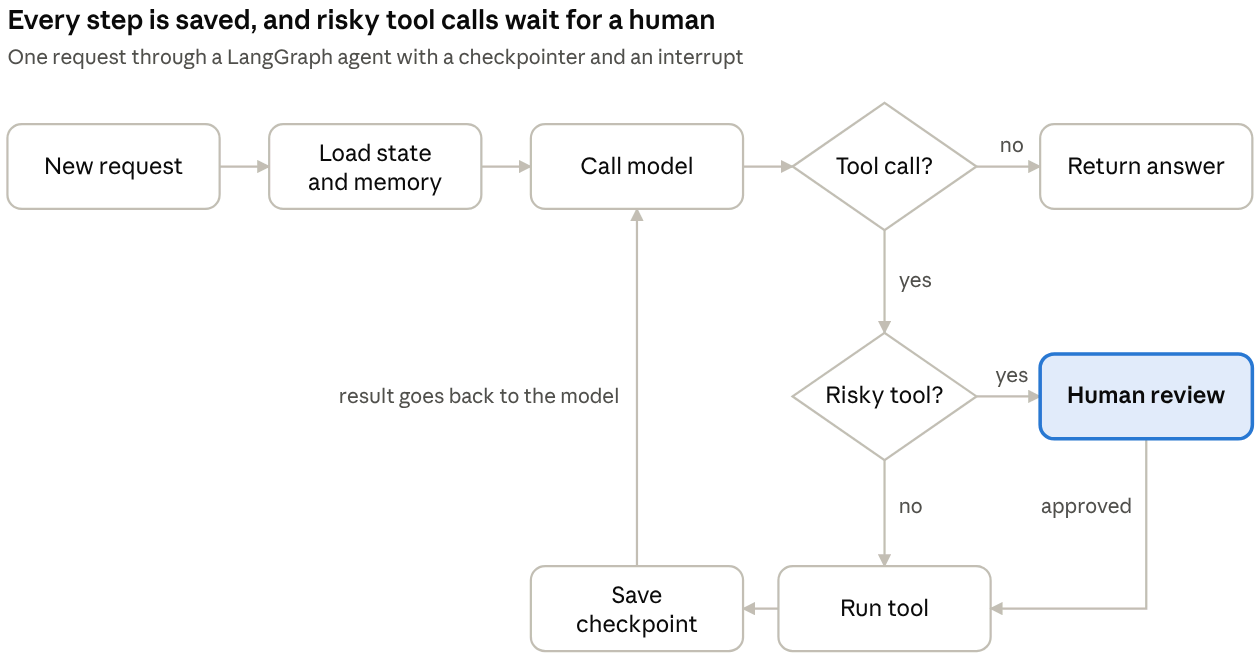
Stage	Main output
Define	Task spec, success metrics, out-of-scope list

Stage	Main output
Design	Architecture choice, tool list, approval points
Build	Prompts, tools, state schema, memory setup
Evaluate	Test dataset, scores, go / no-go decision
Deploy	Running service, guardrails, access controls
Monitor	Traces, dashboards, feedback
Improve	New test cases, revised agent version
Retire	Migration plan, revoked access

A practical rule from the design stage: use the simplest structure that works. A fixed workflow is cheaper, faster and easier to test than an autonomous agent, so autonomy should be added only where the steps genuinely cannot be known in advance.

2. Run-time loop

Every request runs a reason-act loop: the model either asks for a tool or gives the final answer, and tool results feed back until it answers.



run-time loop · reason, act, checkpoint, approve

LangGraph adds two safeguards around the basic loop. A checkpoint after each step means a run that crashes or pauses resumes from the last saved step instead of starting over. An interrupt lets a risky tool call (a payment, an email, a deletion) wait for a person to approve, edit or reject it. Step and cost limits stop the loop if the model never reaches an answer.

3. State and memory

An agent's memory works at three levels, and each needs its own storage and clean-up rules.

Level	What it holds	Lifetime	In LangGraph
Working state	Messages and intermediate results of the current run	One thread	Graph state, saved by the checkpointer per `thread_id`
Long-term memory	User preferences, facts learned, past outcomes	Across threads and sessions	A store, searched and written by the agent
Context window	What the model actually sees on this call	One model call	Built from state and memory on each step

The context window is the constraint that shapes the other two. Long runs fill it, so older messages are summarised or trimmed, and long-term memory is retrieved selectively rather than loaded whole. LangChain's summarisation middleware does the first automatically.

Persistence also changes the lifecycle. Because state outlives a single run, a deployed agent carries data that must be migrated when its state schema changes, and deleted when a user or the agent is retired.

4. Evaluation, observability and guardrails

These three practices span the whole lifecycle rather than belonging to one stage, and together they close the loop from production back to build.

Evaluation checks two things: the outcome (was the answer right, was the task done) and the trajectory (did the agent take a sensible path). Common methods:

- Reference datasets of realistic tasks with expected results, run before every release as a regression suite.
- LLM-as-judge scoring for open-ended answers, calibrated against a sample of human ratings.
- Trajectory checks: which tools were called, in what order, and how many steps it took.
- Online evaluation: scoring a sample of live production runs.

Observability means every run leaves a trace: each model call with its prompt and output, each tool call with its arguments and result, token counts, latency and errors. Without traces, a wrong answer from a ten-step run cannot be debugged. LangSmith provides this for LangChain and LangGraph; OpenTelemetry-based tools cover other stacks.

Guardrails limit damage when the model gets something wrong:

- Least-privilege tools: read-only where possible, scoped credentials, no tool the task does not need.
- Human approval before irreversible actions such as payments, emails or deletions.
- Step, cost and time limits so a looping agent stops.
- Input and output filters for prompt injection, personal data and policy violations.

The improvement loop ties them together: a failure seen in a trace becomes a test case, the fix is evaluated against the whole suite, and the new version is deployed only if nothing else got worse.

Sources

- LangChain and LangGraph Agent Frameworks Reach v1.0 Milestones, LangChain blog, 22 October 2025 (agent loop, middleware, durable state, human-in-the-loop)
- LangChain documentation